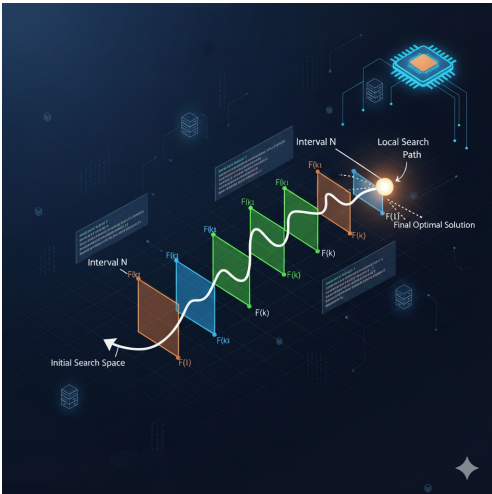




Optimizador de Funciones

Documentación Técnica de Métodos de Optimización

Generado automáticamente — 2026-07-24

1. Introducción

Este documento describe, de forma completa y verificada contra la implementación real del Optimizador, los 32 métodos de optimización disponibles: 8 unidimensionales/de gradiente, 3 metaheurísticos, 6 restringidos (penalización/barrera/suma), y 14 multiobjetivo (4 técnicas clásicas del proyecto + 10 del motor de escalarización).

Cuando un usuario selecciona una combinación método-función incompatible, la aplicación bloquea la ejecución y muestra una explicación en español con métodos alternativos recomendados — ver la matriz de compatibilidad (sección 2).

2. Matriz de compatibilidad función ↔ método

Tipo de función	Compatibles	Incompatibles	Nota
1 variable, cualquier forma (continua, discontinua, no derivable)	Búsqueda Local, SA, PSO, GA	—	Los metaheurísticos y Búsqueda Local no requieren derivadas ni unimodalidad.
1 variable, unimodal en $[l_0, h_1]$	Fibonacci, Sección Áurea, Sección Dorada	Multimodal sin garantía (puede converger a un mínimo local no deseado, sin aviso)	La unimodalidad no se valida automáticamente — es responsabilidad del usuario verificarla.
1 variable o multivariable, diferenciable	Goldstein, Newton-Raphson, Grad. Conjugado, Armijo, Wolfe	Funciones con puntos no derivables en la trayectoria (Abs, sign, Piecewise no suave) — bloqueado con mensaje claro y recomendación de método alternativo	El gradiente/Hessiano se calcula simbólicamente vía SymPy; si sympy no puede derivar, el sistema traduce el error a un mensaje en español.
Multivariable, CON restricción $g(x) \leq 0$ (o $g(x) < 0$ estricta para Barreras)	Los 6 métodos MD (Penalización, Barreras, Pen. BFGS, Pes. BFGS, Pes. Nelder-Mead, Sum. BFGS)	Sin $g(x)$ definido — bloqueado con mensaje y recomendación de método sin restricción	Una sola restricción de desigualdad por corrida; no soporta igualdades ni múltiples restricciones simultáneas.
1 variable (base para $F(x)=[f_1, f_2]$ generado automáticamente)	Los 14 métodos multiobjetivo (Escalarización, MO—Bisección/Sección Dorada/Frente de Pareto/Análisis Jacobiano, y las 9 técnicas del motor de escalarización)	Funciones ya multivariable (2+ variables) — bloqueado con mensaje: «no se pudo aplicar este tipo de estudio a la expresión dada»	El sistema NO acepta un $F(x)$ multivariable directo: siempre parte de $f(x)$ de 1 variable y genera f_2 internamente.

3. Unidimensionales (1D)

Búsqueda Local

Fórmula: Evalúa $f(x)$, $f(x+p)$, $f(x-p)$; se mueve al mejor punto que mejore estrictamente; si ninguno mejora, $p \leftarrow p/2$.

Descripción: Búsqueda de patrón 1D con paso adaptativo (variante simple tipo Hooke-Jeeves), no requiere derivadas.

Requisitos matemáticos: Función evaluable en el rango $[lo,hi]$; no requiere continuidad ni derivabilidad.

Convergencia: Se detiene cuando el paso $p < \text{tolerancia}$ ($1e-5$) o tras 100 iteraciones. Lineal, sensible al paso inicial (0.1).

Ventajas: Muy simple; no necesita derivadas; robusta ante funciones no suaves.

Desventajas / limitaciones: Lenta cerca del óptimo; puede estancarse en funciones con muchos mínimos locales muy cercanos entre sí.

Cuándo usarlo: Introducción a búsqueda 1D; funciones no diferenciables o ruidosas.

Cuándo NO usarlo: Funciones multivariable (solo acepta 1 variable); cuando se necesita velocidad de convergencia alta.

Funciones compatibles: Cualquier función real de 1 variable, continua o no.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje claro).

Ejemplo: $f(x) = x^2 - 4x + 5 \rightarrow x^* \approx 2, f^* \approx 1$

Fibonacci

Fórmula: Reducción de intervalo con razones de Fibonacci: $x_{\text{new}} = a + (F_{N-2}/F_N)(b-a)$, $x_{\text{old}} = a + (F_{N-1}/F_N)(b-a)$.

Descripción: Búsqueda de sección 1D que minimiza el número de evaluaciones de f para una tolerancia objetivo dada.

Requisitos matemáticos: Función unimodal en $[lo,hi]$ (un único mínimo en el intervalo) para garantía teórica de convergencia.

Convergencia: El parámetro L (tolerancia) fija cuántos términos de Fibonacci se generan (objetivo = $(b-a)/L$); óptimamente eficiente en número de evaluaciones para funciones unimodales.

Ventajas: Mínimo número de evaluaciones de f entre los métodos de reducción de intervalo; determinístico.

Desventajas / limitaciones: Requiere unimodalidad — en funciones con varios mínimos puede converger a uno subóptimo sin avisar.

Cuándo usarlo: Funciones costosas de evaluar, unimodales, 1 variable.

Cuándo NO usarlo: Funciones multivariable o claramente multimodales en el rango elegido.

Funciones compatibles: Funciones reales de 1 variable, idealmente unimodales en $[lo,hi]$.

Funciones incompatibles: 2+ variables; L (tolerancia) ≤ 0 (bloqueado con mensaje).

Ejemplo: $f(x) = x^2 - 4x + 5, L=0.1 \rightarrow x^* \approx 2$

Sección Áurea

Fórmula: Igual que Fibonacci pero con razón fija $\tau = (\sqrt{5}-1)/2 \approx 0.618$ en vez de razones de Fibonacci calculadas.

Descripción: Golden-section search: reduce el intervalo en la razón áurea, reutilizando una evaluación de f por iteración.

Requisitos matemáticos: Función unimodal en $[lo,hi]$.

Convergencia: Se detiene cuando el intervalo $(b-a) < 1e-6$, máx. 200 iteraciones. Convergencia lineal con razón $\tau \approx 0.618$.

Ventajas: No necesita precalcular una secuencia como Fibonacci; igual de simple y robusta.

Desventajas / limitaciones: Ligeramente menos eficiente que Fibonacci en evaluaciones totales (óptimo teórico es Fibonacci).

Cuándo usarlo: Alternativa a Fibonacci cuando no se quiere fijar una tolerancia L explícita.

Cuándo NO usarlo: Funciones multivariable; funciones no unimodales.

Funciones compatibles: Funciones reales de 1 variable, unimodales en $[l_0, h_i]$.

Funciones incompatibles: 2+ variables (bloqueado con mensaje claro).

Ejemplo: $f(x) = x^2 - 4x + 5 \rightarrow x^* \approx 2$

Sección Dorada

Fórmula: Misma técnica que «Sección Áurea» (razón áurea τ); en la app es la variante usada además como sub-rutina interna de los métodos multiobjetivo escalarizados.

Descripción: Golden-section search — nombre distinto de «Sección Áurea» en el combo por motivos históricos del proyecto, mismo algoritmo base.

Requisitos matemáticos: Función unimodal en $[l_0, h_i]$.

Convergencia: Igual que Sección Áurea.

Ventajas: Igual que Sección Áurea; además es el solver interno de todas las técnicas de escalarización MO.

Desventajas / limitaciones: Igual que Sección Áurea.

Cuándo usarlo: Igual que Sección Áurea.

Cuándo NO usarlo: Funciones multivariable; no unimodales.

Funciones compatibles: Funciones reales de 1 variable, unimodales.

Funciones incompatibles: 2+ variables.

Ejemplo: $f(x) = x^2 - 4x + 5 \rightarrow x^* \approx 2$

Goldstein

Fórmula: Descenso de gradiente con condiciones de Goldstein: $f_{\text{min}} + \mu\alpha(g \cdot d) \leq f(x + \alpha d) \leq f_{\text{min}} + (1 - \mu)\alpha(g \cdot d)$, $\mu = 0.2$.

Descripción: Búsqueda de línea con doble cota (ni muy poco ni demasiado avance) sobre la dirección de máximo descenso $d = -\nabla f$.

Requisitos matemáticos: f diferenciable (gradiente vía SymPy exacto); acepta 1 o más variables.

Convergencia: Detiene cuando $\|\nabla f\| < 1e-7$; backtracking $\alpha \leftarrow 0.5\alpha$ hasta 50 intentos internos por iteración externa.

Ventajas: Evita pasos demasiado cortos (a diferencia de Armijo puro), mejorando la velocidad práctica.

Desventajas / limitaciones: Usa descenso de gradiente puro (no Newton) — convergencia más lenta que métodos de segundo orden en problemas mal condicionados.

Cuándo usarlo: Funciones diferenciables donde Armijo converge muy lento por pasos pequeños.

Cuándo NO usarlo: Funciones no diferenciables (Abs, signo, etc. en puntos no suaves).

Funciones compatibles: Funciones diferenciables, 1 o más variables.

Funciones incompatibles: Funciones no derivables en la trayectoria de búsqueda.

Ejemplo: $f(x) = x^2 - 4x + 5 \rightarrow x^* \approx 2$

Newton-Raphson

Fórmula: $x_{k+1} = x_k - (H + \text{regl})^{-1} \nabla f(x_k)$, con búsqueda de línea de respaldo si el paso no mejora f .

Descripción: Newton clásico con regularización automática del Hessiano (reg=1e-10, ×10 hasta 8 intentos) y respaldo a descenso de gradiente si sigue singular.

Requisitos matemáticos: f dos veces diferenciable (Hessiano exacto vía SymPy); 1 o más variables.

Convergencia: Cuadrática cerca del óptimo si el Hessiano es bien condicionado; detiene con $\|\nabla f\| < 1e-6$, máx. 50 iteraciones.

Ventajas: Convergencia muy rápida (cuadrática) cerca del mínimo; robusta a Hessianos casi singulares gracias a la regularización.

Desventajas / limitaciones: Cada iteración es más cara (requiere Hessiano); puede alejarse del óptimo si el punto inicial está lejos y la función no es convexa.

Cuándo usarlo: Funciones suaves, dos veces diferenciables, buen punto inicial.

Cuándo NO usarlo: Funciones no diferenciables; alta dimensión (costo del Hessiano crece como n^2).

Funciones compatibles: Funciones C^2 (dos veces diferenciables), 1 o más variables.

Funciones incompatibles: Funciones con puntos no derivables (Abs, Max no suave, etc.) en la trayectoria.

Ejemplo: $f(x) = x^2 - 4x + 5 \rightarrow x^* = 2$ en 1 iteración (cuadrática exacta)

4. Basados en gradiente/Newton (ND, sin restricción)

Grad. Conjugado

Fórmula: Fletcher-Reeves: $\beta_k = \frac{\|g_{k+1}\|^2}{\|g_k\|^2}$; $d_{k+1} = -g_{k+1} + \beta_k \cdot d_k$. Reinicio a $-g$ cada $\max(n, 5)$ iteraciones.

Descripción: Gradiente conjugado no lineal (variante Fletcher-Reeves) con backtracking Armijo ($c_1=1e-4$) para el tamaño de paso.

Requisitos matemáticos: f diferenciable; pensado para multivariable ($n \geq 2$), aunque acepta también 1 variable.

Convergencia: Detiene con $\| \nabla f \| < 1e-6$, máx. 100 iteraciones; en problemas cuadráticos converge en a lo sumo n pasos.

Ventajas: No necesita almacenar/invertir el Hessiano — mucho más barato que Newton en alta dimensión.

Desventajas / limitaciones: Sensible a la precisión de la búsqueda de línea; en problemas no cuadráticos puede perder la propiedad de conjugación sin reinicios frecuentes.

Cuándo usarlo: Problemas multivariable de dimensión media/alta donde el Hessiano es costoso.

Cuándo NO usarlo: Funciones no diferenciables.

Funciones compatibles: Funciones diferenciables, cualquier número de variables.

Funciones incompatibles: Funciones no derivables en la trayectoria.

Ejemplo: $f(x_1, x_2) = (x_1 - 2)^2 + (x_2 - 1)^2 \rightarrow$ converge en pocos pasos

Armijo

Fórmula: Newton con paso de Armijo: $d_k = H^{-1}(-g_k)$ (o $-g_k$ si H singular); acepta α si $f(x + \alpha d) \leq f_k + \mu \cdot \alpha \cdot (g_k \cdot d_k)$, $\mu=0.6$, $\alpha \leftarrow 0.8\alpha$.

Descripción: Newton multivariable con búsqueda de línea de descenso suficiente (condición de Armijo) por retroceso.

Requisitos matemáticos: f diferenciable (gradiente y Hessiano exactos vía SymPy); acepta 1 o más variables.

Convergencia: Detiene con $\| \nabla f \| < 1e-6$, máx. 50 iteraciones; si el Hessiano es singular, detiene inmediatamente (sin respaldo a descenso puro, a diferencia de Newton-Raphson).

Ventajas: Combina la velocidad de Newton con una garantía simple de descenso suficiente; sencillo de razonar.

Desventajas / limitaciones: Sin respaldo si el Hessiano es singular (se detiene ahí, a diferencia de Newton-Raphson que sí tiene fallback); solo controla descenso, no evita pasos demasiado cortos (a diferencia de Goldstein/Wolfe).

Cuándo usarlo: Funciones diferenciables, multivariable, con Hessiano razonablemente bien condicionado.

Cuándo NO usarlo: Funciones no diferenciables (p. ej. Abs en un punto no suave) — el cálculo simbólico de la derivada falla con un error claro.

Funciones compatibles: Funciones diferenciables, cualquier número de variables.

Funciones incompatibles: Funciones no derivables (Abs/sign/Piecewise no suaves en el punto de evaluación).

Ejemplo: $f(x_1, x_2) = 2(x_1 - 3)^2 + x_1 \cdot x_2^3$, con $x_0 = [0, 0]$

Wolfe

Fórmula: Newton con condición de curvatura fuerte: $|\nabla f(x + \alpha d) \cdot d| \leq \mu \cdot |\nabla f(x) \cdot d|$, $\mu=0.6$, resuelta por bisección en $\alpha \in [0, 1]$.

Descripción: Newton multivariable con búsqueda de línea que garantiza curvatura adecuada (evita pasos demasiado cortos), vía bisección — no incluye la condición de descenso suficiente por separado (queda desacoplada de Armijo en esta implementación).

Requisitos matemáticos: f diferenciable (gradiente y Hessiano exactos); acepta 1 o más variables.

Convergencia: Bisección detiene si la condición de curvatura se cumple o el intervalo $(v-u) < 1e-7$, máx. 20 iteraciones internas.

Ventajas: Evita pasos demasiado cortos, favoreciendo avance más agresivo que Armijo puro.

Desventajas / limitaciones: Al no combinar explícitamente la condición de descenso suficiente en el mismo paso, no es un Wolfe estándar de libro de texto (fuerte/débil) sino una variante simplificada centrada en curvatura.

Cuándo usarlo: Funciones diferenciables donde Armijo converge muy lento (pasos demasiado cortos).

Cuándo NO usarlo: Funciones no diferenciables.

Funciones compatibles: Funciones diferenciables, cualquier número de variables.

Funciones incompatibles: Funciones no derivables en la trayectoria.

Ejemplo: $f(x_1, x_2) = 2(x_1 - 3)^2 + x_1 \cdot x_2^3$, con $x_0 = [0, 0]$

5. Metaheurísticos (globales, sin gradiente)

SA: Recocido Simulado

Fórmula: Enfriamiento geométrico $T \leftarrow 0.95 \cdot T$; vecino $x' = x + N(0, 0.3)$; acepta si $\Delta E < 0$, o con prob. $\exp(-\Delta E/T)$.

Descripción: Metaheurística de recocido simulado: $T_{\text{init}}=100$, $T_{\text{min}}=1e-4$, máx. 500 iteraciones, semilla fija (reproducible).

Requisitos matemáticos: Función evaluable en el rango; no requiere continuidad ni derivabilidad.

Convergencia: Probabilística/heurística, no garantizada; detiene por $T < T_{\text{min}}$ o iteraciones agotadas. Guarda el mejor punto visto, no solo el actual.

Ventajas: Escapa de mínimos locales gracias a la aceptación probabilística; robusta ante ruido y discontinuidades.

Desventajas / limitaciones: No determinística en el sentido de garantía de óptimo global; requiere ajustar el enfriamiento para problemas difíciles.

Cuándo usarlo: Funciones con muchos mínimos locales, no diferenciables, o ruidosas.

Cuándo NO usarlo: Cuando se necesita una garantía de convergencia exacta/determinística, o el presupuesto de evaluaciones es muy bajo.

Funciones compatibles: Cualquier función evaluable (continua, discontinua, no diferenciable), 1 o más variables (vía bounds).

Funciones incompatibles: Ninguna restricción estructural — el rango $[l_o, h_i]$ siempre debe ser válido ($l_o < h_i$).

Ejemplo: $f(x) = x^2 - 4x + 5$, rango $[0, 5]$

PSO: Enjambre

Fórmula: $v \leftarrow w \cdot v + c_1 \cdot r_1 \cdot (p_{\text{best}} - x) + c_2 \cdot r_2 \cdot (g_{\text{best}} - x)$, $w=0.7$, $c_1=c_2=1.5$; posición recortada a $[l_o, h_i]$.

Descripción: Particle Swarm Optimization: 30 partículas, 150 iteraciones máx. (config. de la app), semilla fija.

Requisitos matemáticos: Función evaluable en el rango $[l_o, h_i]$; no requiere derivabilidad.

Convergencia: Heurística; detiene antes si $g_{\text{best}} < 1e-10$ o la diversidad del enjambre cae por debajo de $1e-8$.

Ventajas: Buena exploración global; fácil de paralelizar conceptualmente; pocos hiperparámetros.

Desventajas / limitaciones: Puede converger prematuramente en problemas muy multimodales si $w/c_1/c_2$ no están bien ajustados; sin garantía de óptimo global.

Cuándo usarlo: Funciones multimodales, no diferenciables, con rango acotado conocido.

Cuándo NO usarlo: Cuando se requiere precisión muy alta cerca del óptimo (mejor refinar después con un método de gradiente).

Funciones compatibles: Cualquier función evaluable en un rango acotado, 1 o más variables.

Funciones incompatibles: Rango inválido ($l_o \geq h_i$, bloqueado con mensaje).

Ejemplo: $f(x) = x^2 - 4x + 5$, rango $[0, 5]$

GA: Algoritmo Genético

Fórmula: Selección por torneo ($k=3$); cruce aritmético (blend, $\alpha=0.5$); mutación gaussiana adaptativa $\sigma = 0.3(1 - g/\max_gen) + 0.02$; elitismo=3.

Descripción: Algoritmo genético real-valued: población 50, 150 generaciones máx., $p_{\text{cruce}}=0.8$, $p_{\text{mutación}}=0.15$.

Requisitos matemáticos: Función evaluable en el rango $[l_o, h_i]$; no requiere derivabilidad.

Convergencia: Heurística; detiene antes si el mejor $f < 1e-10$ o la diversidad poblacional cae por debajo de $1e-9$.

Ventajas: Muy robusta ante funciones discontinuas, ruidosas o con múltiples óptimos; el elitismo evita perder buenas soluciones.

Desventajas / limitaciones: Puede requerir muchas evaluaciones de f para converger con precisión; sensible a los parámetros de mutación en problemas de alta dimensión.

Cuándo usarlo: Funciones muy irregulares, con múltiples mínimos locales similares.

Cuándo NO usarlo: Presupuesto de evaluaciones muy limitado, o se necesita precisión numérica alta cerca del óptimo.

Funciones compatibles: Cualquier función evaluable en un rango acotado, 1 o más variables.

Funciones incompatibles: Rango inválido ($lo \geq hi$).

Ejemplo: $f(x) = x^2 - 4x + 5$, rango $[0,5]$

6. Restringidos (MD) — penalización / barrera / suma

MD: Penalización (Newton)

Fórmula: $\phi(x)=f(x)+\mu \cdot \max(0,g(x))^2$, $\mu \leftarrow 10\mu$ en 3 pasadas externas ($\mu_{\text{inicial}}=1$); resuelto con Newton propio regularizado por pasada.

Descripción: Penalización exterior cuadrática clásica para $g(x) \leq 0$, con solver interno Newton+Armijo.

Requisitos matemáticos: Requiere $g(x)$ — sin restricción, la penalización no hace nada. Interpreta $g(x) \leq 0$ como restricción de desigualdad; una sola restricción por corrida.

Convergencia: 3 pasadas externas con μ creciendo $\times 10$; cada pasada converge cuando $\|\nabla \phi\| \leq 1e-6$, máx. 60 iteraciones internas.

Ventajas: Simple de entender y depurar; no requiere punto inicial factible (a diferencia de barrera).

Desventajas / limitaciones: Mal condicionamiento numérico cuando μ crece mucho; solo admite $g(x) \leq 0$ (no igualdades, no múltiples restricciones).

Cuándo usarlo: Enseñar/demostrar penalización exterior; punto inicial infactible.

Cuándo NO usarlo: Sin restricción definida (bloqueado); restricciones de igualdad puras o múltiples restricciones simultáneas (no soportado).

Funciones compatibles: f y g diferenciables, 1+ variables, con $g(x)$ definida.

Funciones incompatibles: Sin $g(x)$ (bloqueado con mensaje y recomendación); funciones no diferenciables.

Ejemplo: $f=(x_1-2)^2+(x_2-1)^2$, $g=x_1+x_2-2 \rightarrow x^* \approx [1.5, 0.5]$, $f^* \approx 0.5$

MD: Barreras (Newton)

Fórmula: $\phi(x)=f(x)-(1/t) \cdot \log(-g(x))$, $t \leftarrow 10t$ en 3 pasadas ($t_{\text{inicial}}=1$); requiere punto factible interior $g(x) < 0$.

Descripción: Método de barrera logarítmica (interior point clásico) para $g(x) < 0$ estricta, con solver interno Newton.

Requisitos matemáticos: Requiere $g(x)$. Si x_0 es infactible, la app empuja automáticamente x_0 a lo largo de $-\nabla g$ hasta 50 pasos, o aplica un jitter si no lo logra.

Convergencia: 3 pasadas con t creciendo $\times 10$; cada pasada usa línea de búsqueda que preserva factibilidad (α se reduce hasta que $g(x_{\text{new}}) < 0$), converge con $\|\nabla \phi\| \leq 1e-6$.

Ventajas: Mantiene siempre un punto factible durante toda la trayectoria — útil para visualizar cómo se acerca a la frontera de la restricción.

Desventajas / limitaciones: Requiere estrictamente $g(x) < 0$ (desigualdad estricta); numéricamente más delicado que penalización si el punto inicial está muy cerca de la frontera.

Cuándo usarlo: Cuando importa mantener factibilidad en cada iteración (interpretación tipo interior-point).

Cuándo NO usarlo: Restricciones de igualdad ($g(x)=0$ no tiene interior, la barrera no aplica); sin $g(x)$ definido.

Funciones compatibles: f y g diferenciables, 1+ variables, restricción de desigualdad estricta.

Funciones incompatibles: Sin $g(x)$; restricciones de igualdad.

Ejemplo: $f=(x_1-2)^2+(x_2-1)^2$, $g=x_1+x_2-2 \rightarrow x^* \approx [1.495, 0.495]$, $f^* \approx 0.51$

MD: Pen. (BFGS)

Fórmula: Misma penalización cuadrática que «MD: Penalización (Newton)» ($\phi=f+\mu \cdot \max(0,g)^2$), resuelta con BFGS (scipy) en vez de Newton propio.

Descripción: Compara el efecto del solver interno (BFGS cuasi-Newton vs. Newton exacto) manteniendo fija la técnica de restricción.

Requisitos matemáticos: Requiere $g(x)$. Igual que Penalización (Newton) en la formulación matemática.

Convergencia: 3 pasadas externas, $\mu \times 10$ cada una; BFGS converge superlinealmente sin necesitar el Hessiano exacto.

Ventajas: No requiere Hessiano simbólico (más barato por iteración que Newton en dimensión alta); BFGS suele ser más robusto ante Hessianos mal condicionados.

Desventajas / limitaciones: Convergencia superlineal, no cuadrática — puede necesitar más iteraciones que Newton cerca del óptimo.

Cuándo usarlo: Comparar el efecto del solver interno; problemas donde el Hessiano exacto es costoso o inestable.

Cuándo NO usarlo: Sin $g(x)$ (bloqueado).

Funciones compatibles: f y g diferenciables (BFGS solo necesita gradiente, aproxima el Hessiano), 1+ variables.

Funciones incompatibles: Sin $g(x)$.

Ejemplo: $f=(x_1-2)^2+(x_2-1)^2$, $g=x_1+x_2-2 \rightarrow x^* \approx [1.5, 0.5]$, $f^* \approx 0.5$

MD: Pes. (BFGS)

Fórmula: Penalización LINEAL de peso creciente: $\phi(x)=f(x)+w \cdot \max(0, g(x))$, $w \leftarrow 5w$ en 3 pasadas ($w_{\text{inicial}}=1$), resuelta con BFGS.

Descripción: A diferencia de la penalización cuadrática, aquí el costo de violar la restricción crece linealmente — permite comparar el efecto de la forma de la penalización, no solo del solver.

Requisitos matemáticos: Requiere $g(x)$.

Convergencia: 3 pasadas externas con w creciendo $\times 5$; BFGS converge por pasada.

Ventajas: La penalización exacta lineal puede forzar factibilidad con un peso finito (a diferencia de la cuadrática, que en teoría necesita $\mu \rightarrow \infty$); útil para contrastar ambas formas.

Desventajas / limitaciones: El término lineal introduce un punto no diferenciable en $g(x)=0$ (vía $\max(0, \cdot)$), lo que puede ralentizar la convergencia de un solver basado en gradiente como BFGS cerca de la frontera.

Cuándo usarlo: Comparar penalización lineal vs. cuadrática; funciones acotadas.

Cuándo NO usarlo: Sin $g(x)$ (bloqueado); funciones no acotadas donde un peso pequeño no basta para contener la divergencia.

Funciones compatibles: f y g diferenciables, 1+ variables, idealmente f acotada.

Funciones incompatibles: Sin $g(x)$.

Ejemplo: $f=(x_1-2)^2+(x_2-1)^2$, $g=x_1+x_2-2 \rightarrow x^* \approx [1.5, 0.5]$, $f^* \approx 0.5$

MD: Pes. (Nelder-Mead)

Fórmula: Misma penalización lineal que «MD: Pes. (BFGS)» ($\phi=f+w \cdot \max(0, g)$, $w \leftarrow 5w$ en 3 pasadas), resuelta con Nelder-Mead (sin derivadas, scipy).

Descripción: Compara un solver sin gradiente (simplex de Nelder-Mead) contra uno con gradiente (BFGS), con la misma técnica de restricción.

Requisitos matemáticos: Requiere $g(x)$. No necesita que f o g sean diferenciables (Nelder-Mead solo evalúa la función).

Convergencia: Más lenta y con más iteraciones que los solvers basados en gradiente; sin garantía teórica fuerte en dimensión alta.

Ventajas: Funciona incluso si f o g no son diferenciables en algún punto; no requiere gradiente ni Hessiano.

Desventajas / limitaciones: Más iteraciones/evaluaciones que BFGS; en funciones no acotadas el simplex puede explorar regiones muy alejadas (divergencia) si el peso no crece lo suficiente.

Cuándo usarlo: f o g con puntos no diferenciables; se prefiere no depender de derivadas simbólicas.

Cuándo NO usarlo: Sin $g(x)$; funciones donde ya se sabe que el gradiente exacto está disponible y es barato (mejor usar BFGS/Newton, más eficientes).

Funciones compatibles: Cualquier f y g evaluables (no necesitan ser diferenciables), 1+ variables.

Funciones incompatibles: Sin $g(x)$.

Ejemplo: $f=(x_1-2)^2+(x_2-1)^2$, $g=x_1+x_2-2 \rightarrow x^* \approx [1.5, 0.5]$, $f^* \approx 0.5$

MD: Sum. (BFGS)

Fórmula: Suma ingenua de una sola pasada, sin escalar peso: $h(x)=f(x)+g(x)$, resuelta con BFGS.

Descripción: Contraejemplo pedagógico: a diferencia de penalización/barrera, no hay ninguna pasada externa que aumente el costo de violar la restricción.

Requisitos matemáticos: Requiere $g(x)$; sin él, degenera en un BFGS sin restricción sobre f .

Convergencia: Una sola corrida de BFGS — mucho más rápida, pero sin ninguna garantía de que el resultado respete $g(x) \leq 0$.

Ventajas: Extremadamente simple y rápida; útil para mostrar por qué las técnicas de penalización/barrera con peso creciente son necesarias.

Desventajas / limitaciones: En general NO fuerza la factibilidad — el óptimo de $h(x)=f(x)+g(x)$ casi nunca coincide con el óptimo restringido real, y en funciones no acotadas puede diverger.

Cuándo usarlo: Únicamente con fines demostrativos/comparativos frente a los otros 5 métodos MD.

Cuándo NO usarlo: Cuando se necesita una solución que realmente respete la restricción — usar Penalización o Barrera en su lugar.

Funciones compatibles: f y g diferenciables, 1+ variables (con fines demostrativos).

Funciones incompatibles: Sin $g(x)$ (bloqueado); no debe usarse cuando se requiere factibilidad garantizada.

Ejemplo: $f=(x_1-2)^2+(x_2-1)^2$, $g=x_1+x_2-2 \rightarrow x^* \approx [1.5, 0.5]$, $f^* \approx 0.5$ (coincide aquí porque el problema es simple; no es garantía general)

7. Multiobjetivo — técnicas clásicas (Schaffer genérico)

MO — Bisección

Fórmula: Bisección sobre $\phi'(x, \alpha) = 0$, donde $\phi(x, \alpha) = \alpha \cdot f_1(x) + (1 - \alpha) \cdot f_2(x)$.

Descripción: Técnica clásica del proyecto (previa al motor de escalarización): resuelve la suma ponderada por bisección en vez de sección áurea.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable.

Convergencia: Bisección estándar, lineal, tolerancia configurable.

Ventajas: Determinística, fácil de trazar paso a paso para fines didácticos.

Desventajas / limitaciones: Igual limitación que suma ponderada: no cubre frentes cóncavos.

Cuándo usarlo: Fines didácticos, ver el proceso de bisección aplicado a MO.

Cuándo NO usarlo: Frente cóncavo; funciones multivariable.

Funciones compatibles: $f(x)$ de 1 variable.

Funciones incompatibles: 2+ variables.

Ejemplo: $f(x) = (x-3)^2$, $\alpha=0.5$

MO — Sección Dorada

Fórmula: Igual que Bisección pero resolviendo $\phi(x, \alpha)$ con búsqueda de sección áurea.

Descripción: Variante didáctica de suma ponderada resuelta con sección áurea en vez de bisección.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable.

Convergencia: Lineal, razón áurea.

Ventajas: Igual que Bisección; consistente con el solver usado por el resto del motor de escalarización.

Desventajas / limitaciones: Igual limitación de convexidad que suma ponderada.

Cuándo usarlo: Fines didácticos.

Cuándo NO usarlo: Frente cóncavo; funciones multivariable.

Funciones compatibles: $f(x)$ de 1 variable.

Funciones incompatibles: 2+ variables.

Ejemplo: $f(x) = (x-3)^2$, $\alpha=0.5$

MO — Frente de Pareto

Fórmula: Barrido de $\alpha \in [0, 1]$ en 80 puntos; para cada α , Newton en $\phi'(x, \alpha) = 0$ (hasta 100 iteraciones), registra $(f_1(x^*), f_2(x^*))$.

Descripción: Genera el frente de Pareto completo (no un solo punto) recorriendo todos los pesos α posibles.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable.

Convergencia: Newton por cada punto del barrido: cuadrática localmente; el barrido en sí es exhaustivo, no iterativo.

Ventajas: Da la visión completa del frente, no solo un punto — ideal para elegir un compromiso informado entre f_1 y f_2 .

Desventajas / limitaciones: Más costoso computacionalmente (80 resoluciones de Newton); solo aplica a f_1/f_2 generados desde una función 1D.

Cuándo usarlo: Se quiere explorar todo el frente antes de decidir un compromiso.

Cuándo NO usarlo: Cuando ya se sabe exactamente qué α se necesita (más rápido usar Suma Ponderada directamente).

Funciones compatibles: $f(x)$ de 1 variable.

Funciones incompatibles: 2+ variables.

Ejemplo: $f(x) = (x-3)^2 \rightarrow 80$ puntos del frente (f_1, f_2)

MO — Análisis Jacobiano

Fórmula: $J(x)=[f_1'(x), f_2'(x)]$ en 15 puntos de $[l_0, h_i]$; $\lambda = J_{f_2} / (J_{f_2} - J_{f_1})$; Pareto-óptimo si $0 \leq \lambda \leq 1$.

Descripción: Verifica la condición de estacionariedad de Pareto (KKT 1D) en una malla de puntos, sin resolver una optimización.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable, diferenciable.

Convergencia: No iterativo — evaluación directa del Jacobiano en 15 puntos fijos.

Ventajas: Muy rápido; permite ver directamente qué regiones del dominio son Pareto-óptimas sin resolver un problema de optimización.

Desventajas / limitaciones: Solo evalúa 15 puntos discretos — puede pasar por alto regiones Pareto-óptimas estrechas entre dos puntos de la malla.

Cuándo usarlo: Análisis exploratorio rápido de estacionariedad antes de optimizar formalmente.

Cuándo NO usarlo: Cuando se necesita el óptimo exacto, no solo un diagnóstico.

Funciones compatibles: $f(x)$ de 1 variable, diferenciable.

Funciones incompatibles: 2+ variables; funciones no diferenciables.

Ejemplo: $f(x) = (x-3)^2 \rightarrow$ tabla de 15 puntos con λ y $\blacksquare$

8. Multiobjetivo — motor de escalarización (Fase 6-8)

Escalarización (Suma Ponderada)

Fórmula: $\phi(x) = w_1 f_1(x) + w_2 f_2(x)$, pesos normalizados a partir de α : $w = [\alpha, 1-\alpha]$.

Descripción: Técnica de escalarización más simple e intuitiva; combina $F(x)=[f_1, f_2]$ en un único escalar ponderado.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app genera $f_1=f(x)$ y $f_2=(x-x_{\text{centro}})^2$ automáticamente); resuelto internamente con Sección Dorada.

Convergencia: La de Sección Dorada sobre $\phi(x, \alpha)$: lineal con razón áurea.

Ventajas: Simple, rápida, fácil de interpretar el efecto de α .

Desventajas / limitaciones: No captura frentes de Pareto cóncavos (zonas del frente que la suma ponderada nunca puede alcanzar, sin importar los pesos).

Cuándo usarlo: Frente de Pareto convexo; se necesita un resultado rápido con pesos intuitivos.

Cuándo NO usarlo: Frente de Pareto con regiones cóncavas — usar NBI o Restricción Normal en su lugar.

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje: «no se pudo aplicar este tipo de estudio»).

Ejemplo: $f(x) = (x-3)^2$, $\alpha=0.5$

MO — Lexicográfico

Fórmula: $\phi = f_1 + \epsilon \cdot f_2 + \epsilon^2 \cdot f_3 + \dots$ (prioridad fija $f_1 \gg f_2$, $\epsilon \rightarrow 0$ aproxima el orden lexicográfico exacto).

Descripción: Optimiza estrictamente por orden de prioridad: primero minimiza f_1 sin importar f_2 , salvo por el término ϵ que desempata.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Modela preferencias jerárquicas claras («el costo importa más que el nivel de servicio, sin excepción»).

Desventajas / limitaciones: Ignora casi por completo el objetivo secundario salvo para desempatar — no da control fino sobre el balance entre objetivos.

Cuándo usarlo: Cuando un objetivo es estrictamente más importante que los demás (no hay trade-off aceptable).

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$

MO — Goal Programming

Fórmula: $\phi = \sum w_i \cdot |f_i(x) - \text{meta}_i|^p$ ($p=1$ por defecto).

Descripción: Define una meta aspiracional por objetivo y minimiza las desviaciones ponderadas respecto a esas metas.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Muy intuitivo para el usuario de negocio: «quiero costo $\leq X$ y servicio $\geq Y$ », no pesos abstractos.

Desventajas / limitaciones: El resultado depende fuertemente de qué tan realistas sean las metas elegidas; metas inalcanzables distorsionan la solución.

Cuándo usarlo: Cuando el decisor puede expresar metas concretas por objetivo (satisfacer en vez de optimizar puro).

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$, metas = punto ideal

MO — ε -Constraint

Fórmula: $\phi = f_{\text{primario}}(x) + 1e6 \cdot \sum \max(0, f_i(x) - \varepsilon_i)^2$ ($i \neq \text{primario}$).

Descripción: Deja un objetivo como función a minimizar y convierte los demás en restricciones ($f_i \leq \varepsilon_i$) con penalización cuadrática.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Da control directo sobre el nivel de servicio aceptable; la interpretación es clara y la solución es fácil de justificar.

Desventajas / limitaciones: Requiere elegir ε con cuidado — un ε mal elegido puede volver el problema infactible o trivial.

Cuándo usarlo: Hay un objetivo dominante claro y los demás son restricciones de servicio.

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$, ε calculado a partir de la tabla de pagos

MO — ASF (Logro)

Fórmula: $\phi = \max_i [w_i (f_i(x) - z_i)] + \rho \cdot \sum (f_i(x) - z_i)$ (Wierzbicki, $\rho = 1e-4$).

Descripción: Función de logro (Achievement Scalarizing Function): minimiza la máxima distancia ponderada a un punto de referencia z (típicamente el punto ideal).

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Puede alcanzar cualquier punto del frente de Pareto (incluso en regiones cóncavas), a diferencia de la suma ponderada.

Desventajas / limitaciones: La función $\max(\cdot)$ no es diferenciable en el punto de cambio de objetivo dominante — puede requerir más iteraciones del solver.

Cuándo usarlo: Se quiere explorar todo el frente de Pareto, incluidas regiones cóncavas.

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$, punto de referencia = ideal

MO — Chebyshev

Fórmula: $\phi = \max_i [w_i \cdot |f_i(x) - z_i^*|]$ (z_i^* = punto ideal).

Descripción: Distancia mínima ponderada al punto ideal en norma infinito (Chebyshev ponderado).

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Como ASF, cubre todo el frente de Pareto (cóncavo o convexo) variando los pesos.

Desventajas / limitaciones: No diferenciable en el punto de cambio de objetivo dominante; sensible a la elección del punto ideal.

Cuándo usarlo: Se quiere el punto del frente más cercano (en norma ∞) al ideal, con control de pesos.

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$, punto ideal calculado de la tabla de pagos

MO — NBI

Fórmula: $t(x) = ((F(x) - z) \cdot n) / \|n\|^2$; $\phi = \|F(x) - z\| - t \cdot \|n\|^2 - \eta \cdot t$ (Normal Boundary Intersection simplificado, $\eta=0.1$).

Descripción: Busca la intersección del frente de Pareto con una línea normal al segmento entre los puntos extremos de la tabla de pagos — genera puntos bien distribuidos en el frente.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Distribución más uniforme de puntos en el frente que la suma ponderada, incluso en frentes con curvatura variable.

Desventajas / limitaciones: Requiere la tabla de pagos (payoff table) precomputada; formulación más compleja de interpretar que suma ponderada.

Cuándo usarlo: Se necesita un muestreo bien distribuido del frente de Pareto completo.

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$, tabla de pagos 2×2

MO — Restricción Normal

Fórmula: $\phi = f_{\text{representativo}}(x) + 1e6 \cdot \max(0, (F(x) - z) \cdot n)^2$ (Normal Constraint).

Descripción: Variante de NBI: fija un objetivo representativo y restringe el movimiento a lo largo de la dirección normal mediante penalización.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Como NBI, produce puntos bien distribuidos; más simple de implementar/depurar que NBI puro.

Desventajas / limitaciones: Igual dependencia de la tabla de pagos; el peso de penalización (1e6) puede requerir ajuste fino en problemas mal escalados.

Cuándo usarlo: Alternativa a NBI cuando se prefiere una formulación de penalización en vez de proyección exacta.

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$, tabla de pagos 2×2

MO — Peso Adaptativo

Fórmula: $\phi = \sum w_j f_j(x) - \text{curvature_penalty} \cdot \min_j F(x) - p \sum w_j^2$ (Adaptive Weighted Sum).

Descripción: Extiende la suma ponderada con un término que penaliza la cercanía a puntos vecinos ya encontrados, empujando hacia regiones no exploradas del frente (incluidas cóncavas).

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Puede cubrir regiones cóncavas que la suma ponderada simple no alcanza, sin la formulación $\max(\cdot)$ no diferenciable de ASF/Chebyshev.

Desventajas / limitaciones: Requiere puntos vecinos (`neighbor_points``) para que el término de curvatura tenga efecto — sin ellos, se reduce a suma ponderada simple.

Cuándo usarlo: Se está muestreando el frente iterativamente y se quiere evitar puntos redundantes.

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$, sin puntos vecinos $\rightarrow$ equivale a suma ponderada

MO — Punto de Referencia

Fórmula: $\phi = \max_j [w_j (f_j(x) - z_j)] + p \cdot \sum (f_j(x) - z_j)$ (misma fórmula que ASF, pero z es una aspiración libre, no necesariamente el ideal).

Descripción: Igual formulación matemática que ASF/Logro, pero pensada para que el decisor fije un punto de referencia propio (no necesariamente alcanzable), típico de métodos interactivos de decisión.

Requisitos matemáticos: $f(x)$ mono-objetivo de 1 variable (la app construye $F(x)=[f_1, f_2]$ automáticamente); resuelto con Sección Dorada.

Convergencia: La del solver interno (Sección Dorada): lineal, razón áurea.

Ventajas: Permite exploración interactiva: el decisor mueve el punto de referencia y ve cómo cambia la solución.

Desventajas / limitaciones: Misma no-diferenciabilidad que ASF; el resultado depende fuertemente del punto de referencia elegido.

Cuándo usarlo: Proceso interactivo de decisión donde el usuario ajusta su punto de referencia iterativamente.

Cuándo NO usarlo: Funciones de 2+ variables (el motor de escalarización opera sobre $F(x)=[f_1, f_2]$ generado desde una $f(x)$ de 1 variable, no sobre funciones ya multivariable).

Funciones compatibles: $f(x)$ de exactamente 1 variable.

Funciones incompatibles: Funciones de 2+ variables (bloqueado con mensaje profesional).

Ejemplo: $f(x) = (x-3)^2$, punto de referencia = $f(x_0)$

9. Funciones Matemáticas para Problemas de Inventario

EOQ Clásico (Cantidad Económica de Pedido)

Fórmula: $C(Q) = (D/Q) \cdot S + (Q/2) \cdot H$, $Q^* = \sqrt{(2DS/H)}$

Descripción: Modelo determinista clásico de Harris-Wilson: balancea el costo de ordenar ($D/Q \cdot S$, decrece con Q) contra el costo de mantener inventario ($Q/2 \cdot H$, crece con Q).

Requisitos matemáticos: Variables: $Q > 0$ (cantidad de pedido). Parámetros: D (demanda anual), S (costo fijo por pedido), H (costo de mantenimiento por unidad-año). Sin restricciones activas ni faltantes.

Convergencia: Función convexa 1D con mínimo único — cualquier método de búsqueda 1D o basado en gradiente converge al Q^* analítico.

Ventajas: Fórmula cerrada conocida para validar la convergencia numérica; problema convexo simple.

Desventajas / limitaciones: No modela faltantes, descuentos por volumen ni demanda variable.

Cuándo usarlo: Gestión de inventario con demanda y costos constantes y conocidos.

Cuándo NO usarlo: Cuando hay descuentos por volumen, faltantes permitidos o demanda incierta (usar los otros 3 modelos).

Funciones compatibles: Búsqueda Local, Fibonacci, Sección Áurea, Sección Dorada, Goldstein, Newton-Raphson, Grad. Conjugado, Armijo, Wolfe, y metaheurísticas (SA/PSO/GA).

Funciones incompatibles: Los 6 métodos MD (no hay restricción $g(x)$) y los 14 métodos multiobjetivo (una sola función de costo).

Ejemplo: $D=1000$, $S=50$, $H=2 \rightarrow Q^* \approx 223.6$, $\text{Costo}^* \approx 447.2$

EOQ con Faltantes Planeados (Backorders)

Fórmula: $C(Q,b) = (D/Q) \cdot S + H \cdot (Q-b)^2 / (2Q) + B \cdot b^2 / (2Q)$

Descripción: Extiende el EOQ clásico permitiendo un déficit controlado b , penalizado con un costo de faltante B por unidad-año.

Requisitos matemáticos: Variables: $Q > 0$, $0 \leq b \leq Q$. Parámetros: D , S , H , B (costo de faltante). 2 variables de decisión.

Convergencia: Función convexa en (Q,b) — óptimo analítico conocido para validar la solución numérica.

Ventajas: Reduce el costo total frente al EOQ clásico cuando el faltante es tolerable.

Desventajas / limitaciones: Requiere estimar B (costo de faltante), a menudo más difícil de medir que H .

Cuándo usarlo: Cuando el negocio puede tolerar backorders (pedidos pendientes) sin perder la venta.

Cuándo NO usarlo: Cuando un faltante implica pérdida de venta o cliente (usar EOQ clásico).

Funciones compatibles: Armijo, Wolfe (multivariable) y los 6 métodos MD (Penalización/Barreras/BFGS/Nelder-Mead) — todos aceptan 2 variables.

Funciones incompatibles: Búsqueda Local, Fibonacci, Sección Áurea/Dorada (estrictamente 1D) y los 14 métodos multiobjetivo.

Ejemplo: $D=1000$, $S=50$, $H=2$, $B=8 \rightarrow Q^* \approx 250.0$, $b^* \approx 50.0$

EOQ con Descuentos por Cantidad

Fórmula: $C_i(Q) = (D/Q) \cdot S + (Q/2) \cdot (r \cdot p_i) + D \cdot p_i$, por tramo i

Descripción: Función de costo por tramos: el precio unitario p_i (y por tanto el costo de mantenimiento $r \cdot p_i$) cambia según el rango de cantidad pedida.

Requisitos matemáticos: Variable: $Q > 0$. Parámetros: D , S , tasa de mantenimiento r (fracción del precio), y ≥ 2 tramos (cantidad_mínima, precio_unitario) ordenados.

Convergencia: No derivable en los puntos de quiebre entre tramos — requiere métodos que no dependan de gradiente.

Ventajas: Modela descuentos reales por volumen de compra.

Desventajas / limitaciones: Función no suave; los métodos basados en gradiente pueden fallar o dar resultados incorrectos cerca de los quiebres.

Cuándo usarlo: Cuando el proveedor ofrece precios escalonados por volumen.

Cuándo NO usarlo: Cuando el precio unitario es fijo (usar EOQ clásico).

Funciones compatibles: Búsqueda Local y metaheurísticas (SA: Recocido Simulado, PSO: Enjambre, GA: Algoritmo Genético).

Funciones incompatibles: Newton-Raphson, Goldstein, Armijo, Wolfe, Grad. Conjugado (requieren derivadas continuas); Fibonacci y Sección Áurea/Dorada (asumen unimodalidad, que los saltos de precio pueden romper); los 6 métodos MD y los 14 multiobjetivo.

Ejemplo: $D=1000$, $S=50$, $r=0.2$, tramos= $[(0,10),(100,9),(500,8)] \rightarrow$ compara costo total entre tramos factibles

Revisión Periódica / Punto de Reorden (ROP) Probabilístico

Fórmula: $ROP = \mu_L + z \cdot \sigma_L$, $\mu_L = d \cdot L$, $\sigma_L = \sigma_d \cdot \sqrt{L}$, $\Phi(z) = \text{nivel_servicio}$

Descripción: Calcula el punto de reorden bajo demanda probabilística: stock de seguridad = $z \cdot \sigma_L$, donde z es la inversa de la normal estándar evaluada en el nivel de servicio deseado. Q^* se sigue optimizando con la función de costo del EOQ clásico.

Requisitos matemáticos: Parámetros: d (demanda diaria promedio), σ_d (desviación estándar diaria), L (lead time en días), nivel de servicio $\in (0,1)$, más D , S , H para Q^* .

Convergencia: El ROP se calcula analíticamente (no requiere optimizador); Q^* converge igual que en EOQ clásico.

Ventajas: Incorpora incertidumbre de la demanda con un nivel de servicio objetivo explícito.

Desventajas / limitaciones: Asume demanda normalmente distribuida durante el lead time.

Cuándo usarlo: Cuando la demanda es variable y se requiere un nivel de servicio (probabilidad de no quiebre de stock) explícito.

Cuándo NO usarlo: Cuando la demanda es determinista y constante (usar EOQ clásico).

Funciones compatibles: Mismos métodos que EOQ clásico para optimizar Q (el ROP en sí es analítico, no pasa por el optimizador).

Funciones incompatibles: Los 6 métodos MD y los 14 multiobjetivo (misma razón que EOQ clásico).

Ejemplo: $d=20$, $\sigma_d=5$, $L=7$, nivel_servicio=0.95 $\rightarrow z \approx 1.645$, $ROP \approx 171.9$

10. Implementación del Agente de IA

Esta sección documenta cómo está implementado y funcionando el asistente conversacional dentro de la app (no es una descripción genérica) — ver `app/infrastructure/ai_assistant.py`.

Proveedor y modelo conversacional

El chat usa Groq como proveedor principal (API compatible con OpenAI, gratuita, ~30 solicitudes/minuto) con fallback automático a OpenRouter si Groq falla por red, límite de tasa o error del servicio. El proveedor se detecta por el prefijo de la API key ('gsk_' → Groq, 'sk-or-' → OpenRouter); el usuario puede configurar una key principal y una de respaldo. Los reintentos ante error son silenciosos (el usuario no ve mensajes técnicos) con espera corta creciente (600ms, 1.2s, 1.8s) antes de recurrir al respaldo.

Prompt de sistema y alcance

Un prompt de sistema fijo describe los 32 métodos de optimización disponibles (fórmulas, cuándo usarlos, incompatibilidades) y los 4 modelos del módulo de Inventario (EOQ Clásico, EOQ con Faltantes, EOQ con Descuentos, Punto de Reorden), con reglas explícitas de qué algoritmo recomendar para cada uno y por qué. El asistente restringe sus respuestas al dominio de optimización matemática y esta aplicación; ante preguntas fuera de dominio devuelve un mensaje fijo de redirección.

Contexto dinámico por cálculo

Tras cada cálculo, la ventana principal llama a `AIAssistantPanel.update_context(...)` con el método usado, la función, variables, rango, número de iteraciones y el resultado numérico obtenido — y, si el cálculo fue un modelo de Inventario, también sus valores analíticos (Q^* , costo total, ROP, etc.). Ese contexto se concatena al prompt de sistema en cada mensaje, así que la IA responde con base en lo que realmente se calculó en la sesión, no en una descripción genérica.

Detección de intención de Inventario

Si la respuesta de la IA nombra explícitamente uno de los 4 modelos de Inventario por su etiqueta exacta del combo (coincidencia de texto determinista, no una heurística libre), se emite una señal Qt (`inventory_model_suggested`) que la ventana principal usa para activar ese método automáticamente — el usuario no necesita seleccionarlo a mano.

Modos de respuesta: Escrito / Voz / Ambas

El usuario elige, la primera vez que envía un mensaje, si quiere respuesta por escrito, por voz, o ambas — la elección queda fija para el resto de la sesión (se puede cambiar en cualquier momento con los botones junto al cuadro de texto). En modo Voz o Ambas aparece automáticamente un reproductor compacto (Play/Pausa/Detener); en modo Escrito permanece oculto por completo.

Entrada por voz (Speech-to-Text)

La grabación usa `sounddevice` (16 kHz mono) en un hilo aparte. La transcripción se hace con Groq Whisper (modelo `whisper-large-v3-turbo`); si ese endpoint falla (bloqueo de red, error del proveedor), cae automáticamente a una transcripción local sin red mediante el paquete `SpeechRecognition` (API gratuita de Google Web Speech), usando el mismo WAV ya grabado, sin pedir que el usuario grabe de nuevo. Al terminar de transcribir, la solicitud se envía y procesa automáticamente.

Salida por voz (Text-to-Speech)

La síntesis usa `pyttsx3` sobre el motor SAPI5 de Windows (offline, sin costo ni dependencia de red), con una voz en español instalada en el sistema. Antes de sintetizar, el texto de la respuesta se traduce de notación matemática a palabras naturales (exponentes: x^{**2} → 'x al cuadrado'; subíndices: x_k → 'x sub k'; letras griegas: α → 'alfa'; operadores: $\leq$ → 'menor o igual que', $\sqrt{}$ → 'raíz cuadrada de', etc.) para que se escuche natural en vez de leer

símbolos literalmente. El reproductor usa `Pause()`/`Resume()` nativos del objeto COM de SAPI5 — la pausa/reanudación es real (continúa exactamente donde se cortó), no una resíntesis desde el inicio.

Privacidad y almacenamiento

Las API keys se guardan localmente en la configuración de la app (no en un servidor propio). Las conversaciones no se registran ni envían a ningún servidor del proyecto — solo se transmiten al proveedor de IA (Groq u OpenRouter) que el propio usuario configuró, bajo los términos de esos servicios.